گزارش تحلیلی پروژه

دستیار هوشمند کارهای روزانه

hermes-mini

پروژه‌ی درس مبانی رایانش ابری

مشخصات

نام و نام‌خانوادگی	معصومه صابر
شماره‌ی دانشجویی	۴۰۲۱۲۷۷۰۹۴
درس	مبانی رایانش ابری
استاد حل تمرین	امیر آبرون
عنوان پروژه	hermes-mini، دستیار کارهای روزانه روی تلگرام
تاریخ	آبان ۱۴۰۵

۱ معماری نهایی

کاربر هر چیزی را به فارسی عادی می‌نویسد و هیچ دستور اسلشی وجود ندارد. تلگرام پیام را با webhook به Worker می‌رساند؛ Worker هویت را بررسی می‌کند، از Workers AI یک خط JSON می‌گیرد، آن را اعتبارسنجی می‌کند، روی D1 اجرا می‌کند و جواب را به تلگرام برمی‌گرداند. یک Cron Trigger هم هر دقیقه همان Worker را برای یادآوری‌ها، پیام صبح و خلاصه‌ی شب صدا می‌زند. **فهمیدن منظور با مدل است، ولی اجرا و نوشتن در دیتابیس فقط با کد خودمان انجام می‌شود.**

آدرس زنده: `https://hermes-mini.masoume-hermes.workers.dev`



مؤلفه	نقش در سیستم
Cloudflare Workers	اجرای کد (JavaScript برای webhook و cron)؛ بدون سرور شخصی
D1	پایگاه‌داده‌ی رابطه‌ای: جدول‌های tasks، chat_history و state با ۲ migration جدا از کد
Workers AI	فهمیدن متن آزاد و تبدیل آن به JSON؛ بدون API key و binding
Cron Trigger	یادآوری سر ساعت، پیام صبح (ساعت ۸) و خلاصه‌ی شب (ساعت ۲۱)
Secret store و Observability	نگهداری توکن بات و رمز webhook؛ لاگ زنده با `wrangler tail`

۲ تصمیم‌های اصلی طراحی و دلیلشان

webhook، نه polling

polling یعنی فرایندی همیشه‌روشن که مدام از تلگرام بپرسد؛ یعنی سرور یا حلقه‌ی دائمی. با webhook تلگرام خودش پیام را می‌فرستد و کد فقط همان لحظه اجرا می‌شود. چون تلگرام در صورت پاسخ غیر ۲۰۰ پیام را دوباره می‌فرستد، بعد از احراز هویت همیشه ۲۰۰ برمی‌گردانیم.

چرا FaaS

بار کاری یک کاربر است و پراکنده؛ نگه‌داشتن VM یا Docker برای آن هدررفت است. Worker فقط هنگام رویداد اجرا می‌شود، روی پلن رایگان می‌گنجد و نگهداری سرور (به‌روزرسانی، امنیت، restart) ندارد.

چرا D1 (رابطه‌ای) و نه KV

داده‌ها ساختار و پرس‌وجو دارند: وضعیت، سررسید، ترتیب، کوچک‌ترین شماره‌ی خالی، کارهای سررسیدشده. در SQL هرکدام یک کوئری است و batch چند نوشتن را اتمیک می‌کند؛ در KV باید همه را خودمان بخوانیم و مرتب کنیم.

state کجا زندگی می‌کند

همه‌چیز در D1: کارها، حافظه‌ی کوتاه مکالمه و state های کوچک (آخرین پیام صبح و شب، تأیید حذف). خود Worker هیچ state ای ندارد؛ پس بعد از هر restart و از هر دستگاه همان داده دیده می‌شود.

۳ هوش مصنوعی، امنیت و زمان

متن آزاد ← **مدل (LLM)** ← **اعتبارسنجی** ← **اجرا روی D1**

- **متن آزاد**: فارسی؛ ارقام و «ی/ک» نرمال می‌شود
- **مدل (LLM)**: فقط یک خط JSON؛ temperature = 0
- **اعتبارسنجی**: شماره‌ها، ساعت و اکشن مجاز چک می‌شود
- **اجرا روی D1**: فقط اگر معتبر بود؛ وگرنه جواب امن

- **اعتبارسنجی خروجی مدل:** اکشن باید از مجموعه‌ی بسته‌ی `add`، `done`، `undone`، `del`، `rename`، `settime`، `list`، `chat` باشد؛ شماره‌ی ساختگی بی‌اثر است، «همه» را کد حساب می‌کند و ساعت را مدل نمی‌سازد.
- **مسیر امن:** خروجی خراب یعنی پیام «متوجه نشدم» و هیچ نوشتنی در دیتابیس. پیام مبهم حذف نمی‌کند و می‌پرسد؛ حذف چند تسک فقط با «تایید» انجام می‌شود. تسک تکراری ثبت نمی‌شود و ۴ پیام آخر حافظه‌ی مکالمه است.
- **امنیت:** درخواست `POST /webhook` بدون هدر secret درست ۴۰۳ می‌گیرد (با `curl` روی آدرس زنده تست شد)؛ فقط یک شناسه‌ی تلگرام مجاز است و بقیه بی‌صدا رد می‌شوند؛ بعد از احراز هویت همیشه ۲۰۰ برمی‌گردد. `BOT_TOKEN` و `WEBHOOK_SECRET` فقط در secret store هستند و `GET /health` سلامت سرویس را نشان می‌دهد.
- **«امروز» به وقت تهران:** سرور منطقه‌ی زمانی ندارد و دیتابیس UTC است؛ پس همه‌ی تاریخ و ساعت‌ها با `Asia/Tehran` و `Intl.DateTimeFormat` حساب می‌شوند. سررسید به شکل تاریخ و ساعت کامل ذخیره می‌شود («فردا ساعت ۸» هم کار می‌کند) و لیست با تاریخ شمسی نمایش داده می‌شود.
- **کارهای خودکار (cron هر دقیقه):** یادآوری سر ساعت (حداقل یک بار: اول ارسال، بعد علامت‌گذاری)، پیام صبح از ساعت ۸ و خلاصه‌ی شب ساعت ۲۱. علامت «امروز فرستاده شد» فقط بعد از ارسال موفق ذخیره می‌شود؛ پس تکرار نمی‌شود و اجرای ازدست‌رفته جبران می‌شود.

۴ تحلیل هزینه در دو مقیاس

مورد	۱ کاربر (≈ ۳۰ پیام در روز)	۱۰٬۰۰۰ کاربر (۲۰ پیام در روز برای هر نفر)
Workers و D1	کاملاً داخل پلن رایگان (≈ ۱٬۵۰۰ اجرا و کمتر از ۵۰ هزار خواندن D1 در روز)	≈ ۶ میلیون اجرا و ≈ ۶۰۰ میلیون خواندن D1 در ماه؛ داخل سهمیه‌ی پلن Paid (۵ دلار در ماه)

Workers AI با مدل 70B (فعلی)	≈ ۱٬۷۰۰ neuron در روز از ۱۰٬۰۰۰ neuron رایگان	≈ ۱۱٫۲ میلیون neuron در روز ⇐ ≈ ۳٬۷۰۰ دلار در ماه
Workers AI با مدل 8B	≈ ۲۶۰ neuron در روز (رایگان)	≈ ۱٫۷۶ میلیون neuron در روز ⇐ ≈ ۵۸۰ دلار در ماه
جمع ماهانه	۰ دلار	≈ ۳٬۷۰۵ دلار با مدل 70B ≈ ۵۸۵ دلار با مدل 8B

فرض‌ها: قیمت‌ها از صفحه‌های رسمی Pricing کلودفلر (مهر ۲۰۲۶) آمده‌اند. هر پیام را حدود ۱۸۰۰ توکن ورودی و ۴۰ توکن خروجی فرض کرده‌ام، یعنی حدود ۵۶ neuron با مدل 70B و ۹ neuron با مدل 8B. **نتیجه:** برای یک کاربر همه‌چیز رایگان است. در ۱۰٬۰۰۰ کاربر بیش از ۹۹٪ هزینه مال استنتاج مدل است، پس انتخاب مدل و کوتاه‌کردن پرامپت مهم‌ترین اهرم هزینه است. نسخه‌ی فعلی تک‌کاربره است؛ برای ۱۰٬۰۰۰ کاربر باید `user_id` و index به جدول‌ها اضافه و ارسال گروهی پیام با Queues تقسیم شود.

۵ کجا گیر کردم و چطور حلش کردم

۱) بات جواب نمی‌داد و خطایی دیده نمی‌شد

webhook سالم بود و Worker پیام را می‌گرفت، ولی جوابی به تلگرام نمی‌رسید و کد خطاها را بی‌صدا قورت می‌داد. با لاگ کردن status و متن پاسخ تلگرام (بدون چاپ توکن) دیدم `sendMessage` خطای ۴۰۴ می‌دهد؛ بعد از بازنویسی تابع ارسال و ذخیره‌ی دوباره‌ی secret درست شد. علت دقیق را قطعی نمی‌دانم (احتمالاً رشته‌ی آدرس یا مقدار توکن).

۲) فیلترشکن مرورگر برای ترمینال کار نمی‌کرد

`npm install` خطای ECONNRESET می‌داد و `wrangler tail` مدام قطع و وصل می‌شد، در حالی که مرورگر مشکلی نداشت. با روشن‌کردن فیلترشکن و باز کردن ترمینال جدید، تست آدرس زنده در مرورگر و استفاده از لاگ داشبورد (Observability) حل شد.

۳) مدل B۸ در پیام‌های سخت ناپایدار بود

«همه رو حذف کن» گاهی همه و گاهی فقط یکی را برمی‌داشت و یک جواب اشتباه به حافظه‌ی مکالمه می‌رفت و تکرار می‌شد. اعتبارسنجی جلوی حذف اشتباه را می‌گرفت، ولی رفتار پایدار نبود؛ راه‌حل: «همه» را کد از دیتابیس حساب می‌کند، مثال‌های بیشتر در پرامپت، حافظه‌ی کوتاه‌تر و رفتن به مدل B۷۰.